

Linux Privilege Escalation: Kernel Exploits

Your Writable Files guide closed by flagging this as the genuine category break in the series — every technique so far (SUID, Capabilities, Cron Jobs, PATH Hijacking, NFS, Writable Files) exploited a **misconfiguration**: something that could, in principle, be fixed by correctly setting a permission, path, or export flag. Kernel exploits target something categorically different — an actual **bug in the kernel's own code**, present regardless of how perfectly everything else on the system is configured. Worth locking in the distinction immediately: **Part 1 covers how kernel vulnerabilities actually work and why they're so severe — Part 2 covers the practical workflow of identifying and safely using a kernel exploit**, since this technique carries meaningfully higher operational risk (system crashes, instability) than anything else in this series, making the "how to do this safely" half at least as important as the "how it works" half.

1. Why a Kernel Bug Is Different From Everything Else in This Series — The Foundation

Every earlier guide's exploitation target ran in **userspace**, as a process subject to the kernel's own permission enforcement — a SUID binary, a cron job, an NFS-mounted file all still had to go THROUGH the kernel's permission checks to do anything privileged. The kernel itself is what enforces every one of those checks.

Userspace privilege escalation (every earlier guide): attacker process → asks kernel to do something → kernel CHECKS permissions → kernel ALLOWS or DENIES based on UID/GID/capabilities/ ACLs → attacker exploits a GAP in how those checks were CONFIGURED

Kernel exploitation (this guide): attacker process → triggers a BUG directly in the kernel's OWN code (a memory corruption, a logic error, a race condition) → the permission-checking mechanism ITSELF is subverted or bypassed entirely → no configuration was ever wrong; the enforcement mechanism's IMPLEMENTATION was flawed

The single fact that makes this category so severe — the kernel runs with unrestricted hardware-level privilege (ring 0 / supervisor mode) and mediates literally every security boundary on the system, so a successful kernel exploit doesn't escalate to root within the existing rules — it breaks

the rules-enforcement mechanism itself, which is why kernel vulnerabilities are consistently rated among the most severe of any vulnerability class regardless of how it's ultimately triggered.

PART 1: Understanding Kernel Vulnerability Classes

2. Memory Corruption Bugs — Buffer Overflows, Use-After-Free

The most classic and historically common kernel bug category — a flaw in the kernel's memory-handling code (a driver, a syscall handler, a filesystem implementation) that lets a userspace process corrupt kernel memory in a controlled way:

```
Buffer overflow:  a kernel structure/buffer receives more data than
                  it was sized for, overwriting ADJACENT kernel
                  memory - including, with enough control, memory
                  holding the current process's own PRIVILEGE
                  CREDENTIALS structure

Use-after-free:   kernel memory is freed, but a dangling reference
                  to it remains usable - an attacker races to
                  reallocate that same memory region with
                  ATTACKER-CONTROLLED content before the kernel
                  uses the dangling reference again
```

Most public kernel privilege-escalation exploits ultimately aim at the same target regardless of the specific initial bug: overwriting the current process's credential structure (`struct cred` in Linux) to directly set its UID/GID to 0, or hijacking a kernel-mode code pointer to redirect execution to attacker-supplied code running with full kernel privilege.

3. Race Conditions — Timing-Dependent Bugs

A specific memory-corruption subcategory worth its own mention given how common it is in real CVEs — a bug that only manifests when two kernel code paths execute in an unexpected order/overlap, typically triggered by deliberately running multiple threads/processes that race against a narrow timing window:

```
Dirty COW (CVE-2016-5195) - one of the most famous Linux kernel
privilege escalation vulnerabilities - exploited a race condition
in the kernel's copy-on-write memory handling, allowing a
```

```
process to write to memory-mapped files it should only have
READ access to, including files it doesn't own, by racing a
madvise() call against a concurrent write
```

4. Logic Bugs — Flawed Permission-Check Implementation

Distinct from memory corruption entirely — a bug where the kernel's permission-checking CODE is simply written incorrectly, with no memory safety issue involved at all, just an implementation mistake in the logic itself:

```
Example pattern: a syscall or driver ioctl handler that checks
permissions correctly on ONE code path but has an alternate
code path (an edge case, a specific flag combination, a
symlink-following quirk) where the check is accidentally
skipped or incorrectly evaluated entirely
```

5. Kernel Module / Driver-Specific Vulnerabilities

Third-party or vendor-specific kernel modules (graphics drivers, VPN client kernel components, virtualization drivers) frequently receive far less security scrutiny than the mainline kernel itself, and are a disproportionately common source of exploitable bugs on systems that have any such module loaded — worth explicitly checking `lsmod` output against known vulnerable module versions, not just the base kernel version.

PART 2: Practical Identification & Exploitation Workflow

6. Identifying the Exact Kernel Version and Distribution Context

```
uname -a
cat /etc/os-release
cat /proc/version
```

```
uname -a output example:
Linux target-host 5.4.0-42-generic #46-Ubuntu SMP ...
```

Precise version matters enormously here - kernel CVEs are almost always fixed in a SPECIFIC version, meaning exploit applicability is a narrow version-range question, not a general "old kernel = vulnerable" heuristic. A kernel one point-release newer than a

known-vulnerable version is frequently NOT exploitable via that same CVE at all.

7. Matching the Kernel Version to Known CVEs

```
# manual research approach:
searchsploit linux kernel 5.4
searchsploit ubuntu 20.04 privilege escalation

# automated matching (covered in depth in the LinPEAS guide
next
# in this series, but worth knowing standalone tools exist
too):
# linux-exploit-suggester, linux-smart-enumeration both cro
ss-
# reference uname output against a maintained CVE database
```

Prioritize LOCALLY WELL-DOCUMENTED, widely-tested exploits (Dirty COW, DirtyPipe, PwnKit/CVE-2021-4034, and similar "famous" kernel CVEs with extensive public writeups and stable, well-tested public PoC code) over obscure or recently-disclosed CVEs with sparse or unverified exploit code - reliability matters enormously more here than novelty, given Section 9's stability concerns

8. Compiling and Running a Public Exploit

```
# typical workflow for a C-based kernel exploit PoC:
gcc exploit.c -o exploit -lpthread # kernel exploits freq
ually

# require specific
compiler

# flags/linked libr
aries -

# ALWAYS read the e
xploit's

# own documentatio
n/comments

# rather than assum
```

```
ing a                                     # generic compile c
ommand works
./exploit
```

Cross-compilation considerations: if the target's architecture or available toolchain differs from your attacking machine, you may need to compile ON the target directly (if a compiler is present) or cross-compile a matching binary beforehand - a genuinely common practical obstacle specific to this technique that doesn't arise in any earlier guide in this series, since none of them required compiling and running NEW attacker-authored machine code at all

9. The Critical Difference From Every Earlier Guide — System Stability Risk

EVERY technique in your SUID, Capabilities, Cron Jobs, PATH Hijacking, NFS, and Writable Files guides carries essentially ZERO risk of crashing or destabilizing the target system - misconfigured permissions being exploited doesn't touch the kernel's own memory safety at all.

Kernel exploits are FUNDAMENTALLY DIFFERENT here - a failed or even a SUCCESSFUL kernel exploit can:

- Crash the target system entirely (kernel panic)
- Corrupt filesystem state or in-memory data mid-write
- Leave the system in a subtly unstable state that fails LATER, even if the immediate exploit attempt appeared to succeed

This is the single most important operational consideration distinguishing this guide from every other one in the series, and directly drives Section 10's methodology and Section 12's authorization guidance.

10. Reducing Risk — Snapshot, Test, and Timing Discipline

- If ANY snapshot/rollback capability exists for the target (a VM snapshot, a cloud instance image, a documented backup), use it immediately before attempting a kernel exploit - non-negotiable standard practice for this technique specifically
- Test the EXACT exploit against a matching kernel version in an isolated lab environment FIRST whenever at all possible, rather than running an untested PoC directly against a live production or client engagement target
- Avoid running kernel exploits during business-critical hours on any production-adjacent system, given the crash risk (Section 9)

- Read the exploit's source/documentation for known caveats before running it - many public PoCs include explicit notes about specific kernel configs, architectures, or mitigations (KASLR, SMEP/SMAP) that affect reliability, and running blind against an untested combination meaningfully raises crash risk

11. Testing Methodology

1. Identify the precise kernel version and distro (Section 6) - never assume, always confirm directly on the target
2. Cross-reference against known CVEs (Section 7), prioritizing well-documented, widely-tested exploits over obscure ones
3. Check for vulnerable third-party kernel modules (Section 5) alongside the base kernel version check, since these are frequently overlooked
4. Where possible, replicate the target's EXACT kernel version in an isolated lab and test the exploit there first (Section 10)
5. Ensure a snapshot/rollback path exists before running against the actual target
6. Run the exploit, and IMMEDIATELY verify the resulting shell (id, whoami) - kernel exploits can sometimes produce a partially-successful or unstable result that looks like success at a glance but isn't fully reliable

12. Legal & Ethical Note

The elevated crash/stability risk covered in Sections 9-10 makes EXPLICIT client authorization and risk acceptance for kernel-level exploitation especially important, beyond the baseline authorization requirement shared with every technique in this series - a kernel panic on a production system, even during an authorized engagement, can cause genuine business disruption (unplanned downtime, potential data loss on unclean shutdown) that the client needs to have explicitly accepted as a possible outcome BEFORE this technique is attempted, not discovered after the fact.

13. Prevention

- Patch kernels promptly and on a regular, defined cadence - the single most direct mitigation, since this entire technique category depends on running a kernel version with a KNOWN, unpatched vulnerability
- Enable and maintain kernel hardening features where available - KASLR (kernel address space layout randomization), SMEP/SMAP (preventing kernel-mode execution of/access to userspace memory),

and similar mitigations don't eliminate underlying bugs but substantially raise the difficulty of reliably exploiting many memory-corruption-class vulnerabilities (Section 2)

- Minimize the set of loaded third-party kernel modules (Section 5) to only what's genuinely required, and keep those patched with the same discipline as the base kernel itself
- Use kernel live-patching solutions where operationally feasible, reducing the window between a CVE's disclosure and the target system actually receiving the fix, without requiring a disruptive reboot for every single patch cycle
- Layer defense - since this technique bypasses userspace permission checks entirely, treat kernel patching as a NECESSARY complement to (not a replacement for) the configuration-hardening measures covered in every earlier guide in this series, rather than assuming strong userspace hardening alone is sufficient

14. Practical Lab Example

Deliberately vulnerable lab VMs running specific older kernel versions (many available as pre-built VulnHub/HTB images specifically built around a named CVE like Dirty COW or DirtyPipe) are the standard way to practice this safely - NEVER practice against a system you don't fully control and haven't snapshotted, given Section 9's stability risk.

Suggested practice order:

1. Set up a VM running a kernel version with a well-known, well-documented CVE (Dirty COW's target kernel range is a common, well-trodden starting point given its extensive public documentation)
2. Snapshot the VM FIRST (Section 10), then compile and run the corresponding public PoC exploit
3. Deliberately run the SAME exploit against a patched, newer kernel version afterward and confirm it fails cleanly - useful for building the instinct that exploit success is tightly version-bound, not a general property of "old-looking" systems

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-787 (Out-of-bounds Write) / CWE-416 (Use After Free) — the two most common memory-corruption root causes underlying real kernel PrivEsc CVEs (Section 2), CWE-362 (Concurrent Execution using Shared Resource with Improper Synchronization — Race Condition, covering Section 3)

Framework	Mapping
OWASP Top 10:2025	Not directly applicable — OWASP's Top 10 targets web application vulnerabilities; kernel exploitation sits outside that scope entirely and is more directly addressed by CWE/CVE frameworks and vendor-specific security advisories
CVSS	Critical, near-universally (AV:L typically, PR:L, but C:H/I:H/A:H given full kernel-level compromise) — kernel privilege escalation CVEs are consistently among the highest-scored entries in any given year's vulnerability disclosures
Cyber Kill Chain	Privilege Escalation, though notably this technique can ALSO directly cause unintended effects landing in Impact/Denial-of-Service territory if an exploit attempt fails and crashes the system (Section 9) — a genuine dual-outcome risk profile unique to this guide in the series
MITRE ATT&CK	T1068 (Exploitation for Privilege Escalation) — its own dedicated technique, distinct from every configuration-based technique ID referenced in earlier guides in this series

Quick Reference Cheat Sheet

IDENTIFY:

```
uname -a ; cat /etc/os-release ; cat /proc/version
lsmod                                     → check third-party modules too
```

RESEARCH:

```
searchsploit linux kernel <version>
Prioritize well-documented, widely-tested CVEs (Dirty COW,
DirtyPipe, PwnKit) over obscure/unverified ones
```

BEFORE RUNNING (non-negotiable given Section 9's crash risk):

- ☐ Snapshot/backup taken
- ☐ Tested against a **MATCHING** kernel version in an isolated lab
- ☐ Explicit client authorization for potential instability/crash

RUN:

```
gcc exploit.c -o exploit [flags per exploit's own documentation]
./exploit
id ; whoami    → immediately verify actual success
```

Root lesson: this is the **ONE** technique in this series that isn't a misconfiguration - it's an actual bug in the kernel's own code, and that categorical difference is exactly why it carries a real stability/crash risk none of the earlier guides' techniques share, and why patching (not permission-tightening) is its specific, non-substitutable defense.